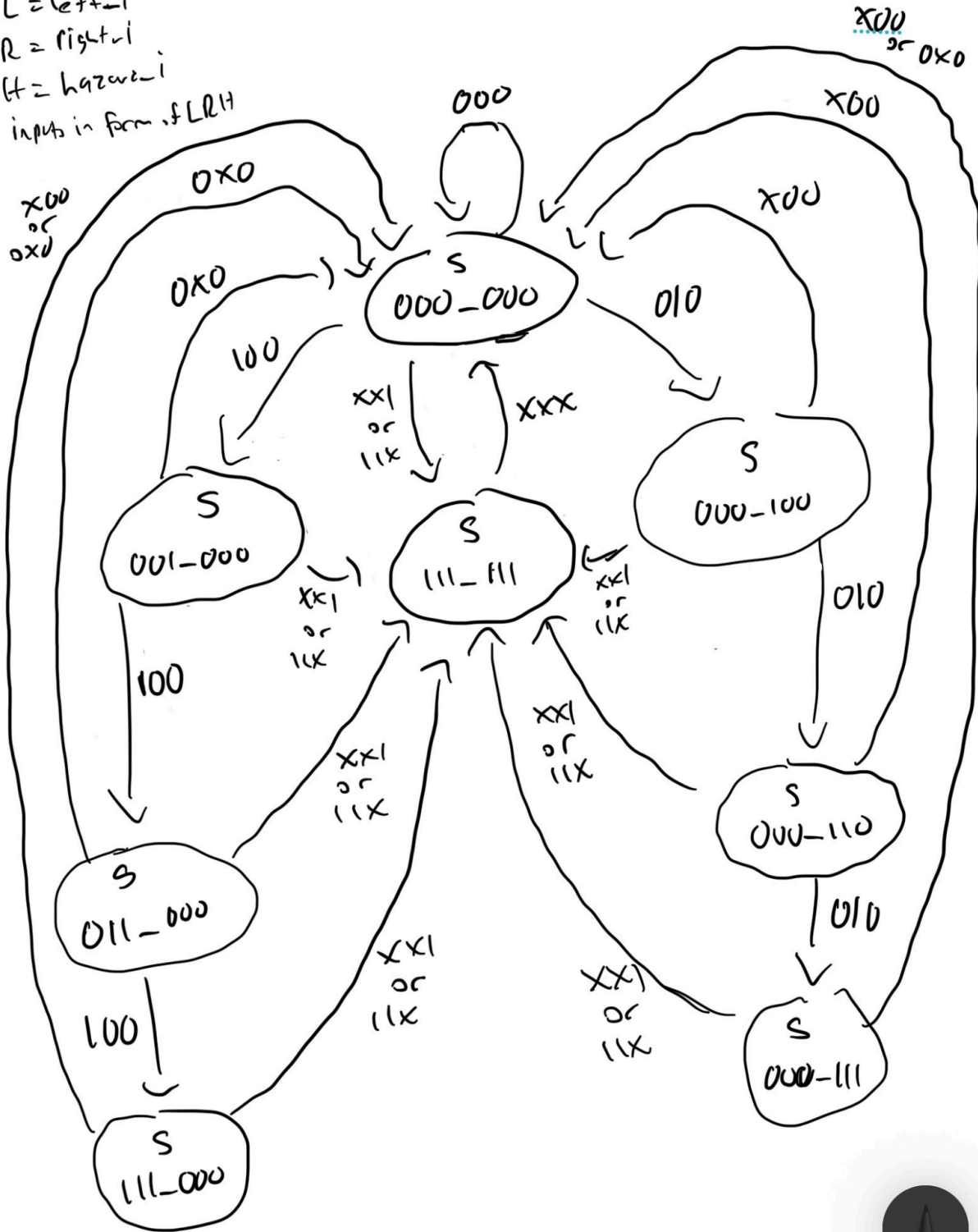


Part 1: Moore state diagram for controller

L = left-
R = right-
H = hazard-
inputs in form of LRH



Part 2: Verilog code and Modelsim simulation waveforms from testbench
usbece152a_fsm.sv:

```
/*
 * Copyright (c) 2023, University of California; Santa Barbara
 * Distribution prohibited. All rights reserved.
 *
 * File: ucsbece152a_fsm.sv
 * Description: Starter code for fsm.
 */
import taillights_pkg::*;
module ucsbece152a_fsm (
    input logic clk,
    input logic rst_n,

    input logic left_i,
    input logic right_i,
    input logic hazard_i,

    output state_t state_o,
    output logic [5:0] pattern_o
);

state_t state_d, state_q = S000_000;
assign state_o = state_q;

// TODO: Implement the FSM and drive `pattern_o`

always_comb begin
    case (state_q)
        S000_000:
            if (!left_i && !right_i && !hazard_i) begin
                state_d = S000_000;
            end
            else if (hazard_i || (left_i && right_i)) begin
                state_d = S111_111;
            end
            else if (left_i && !right_i && !hazard_i) begin
                state_d = S001_000;
            end
            else begin
```

```

        state_d = S000_100;
    end
    S000_100:
    if (!right_i && !hazard_i) begin
        state_d = S000_000;
    end
    else if (hazard_i || (left_i && right_i)) begin
        state_d = S111_111;
    end
    else begin
        state_d = S000_110;
    end
    S000_110:
    if (!right_i && !hazard_i) begin
        state_d = S000_000;
    end
    else if (hazard_i || (left_i && right_i)) begin
        state_d = S111_111;
    end
    else begin
        state_d = S000_111;
    end
    S000_111:
    if (!hazard_i && (left_i || right_i)) begin
        state_d = S000_000;
    end
    else begin
        state_d = S111_111;
    end
    S001_000:
    if (!left_i && !hazard_i) begin
        state_d = S000_000;
    end
    else if ((left_i && right_i) || hazard_i) begin
        state_d = S111_111;
    end
    else begin
        state_d = S011_000;
    end
    S011_000:

```

```

        if (!left_i && !hazard_i) begin
            state_d = S000_000;
        end
        else if ((left_i && right_i) || hazard_i) begin
            state_d = S111_111;
        end
        else begin
            state_d = S111_000;
        end
        S111_000:
        if (!hazard_i && (left_i || right_i)) begin
            state_d = S000_000;
        end
        else begin
            state_d = S111_111;
        end
        S111_111:
        state_d = S000_000;

    endcase
end

always_comb begin
    case (state_q)
        S000_000: pattern_o = 6'b000_000;
        S000_100: pattern_o = 6'b000_100;
        S000_110: pattern_o = 6'b000_110;
        S000_111: pattern_o = 6'b000_111;
        S001_000: pattern_o = 6'b001_000;
        S011_000: pattern_o = 6'b011_000;
        S111_000: pattern_o = 6'b111_000;
        S111_111: pattern_o = 6'b111_111;
    endcase
end

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state_q <= S000_000;
    end else begin
        state_q <= state_d;
    end
end

```

```
        end
    end

endmodule
```

ucsbece152a_taillights.sv:

```
/*
 * Copyright (c) 2023, University of California; Santa Barbara
 * Distribution prohibited. All rights reserved.
 *
 * File: ucsbece152a_taillights.sv
 * Description: Starter code for taillights.
 */
module ucsbece152a_taillights (
    input logic clk,
    input logic rst_n,
    input logic clk_dimmer_i,

    input logic left_i,
    input logic right_i,
    input logic hazard_i,
    input logic brake_i,
    input logic runlights_i,

    output logic [5:0] lights_o
);

logic [5:0] fsm_pattern;

ucsbece152a_fsm fsm (
    .clk(clk),
    .rst_n(rst_n),
    .left_i(left_i),
    .right_i(right_i),
    .hazard_i(hazard_i),
    .pattern_o(fsm_pattern)
);
```

```
logic [5:0] lights_runlightsoff, lights_runlightson;
// TODO: Convert `fsm_pattern` into `lights_o`

assign lights_runlightson = (lights_runlightsoff | {6{clk_dimmer_i}});

assign lights_o = runlights_i ? lights_runlightson : lights_runlightsoff;

always_comb begin
    lights_runlightsoff = (brake_i) ? 6'b111_111 : 6'b000_000;
    if (hazard_i || (left_i && right_i)) begin
        lights_runlightsoff |= fsm_pattern;

    end

    else if (right_i) begin
        lights_runlightsoff[2:0] = fsm_pattern[2:0];

    end

    else if (left_i) begin
        lights_runlightsoff[5:3] = fsm_pattern[5:3];
    end
end

endmodule
```

ucsbece152a_taillights_tb.sv:

```
module ucsbece152a_taillights_tb();

    logic clk = 0;
    always #(100) clk = ~clk;

    logic rst_n;
    logic clk_dimmer_i = 0;
    always #(10) clk_dimmer_i = ~clk_dimmer_i;

    logic left_i;
    logic right_i;
    logic hazard_i;
    logic brake_i;
    logic runlights_i;

    logic [5:0] lights;

    ucsbece152a_taillights DUT (
        .clk(clk),
        .clk_dimmer_i(clk_dimmer_i),
        .rst_n(rst_n),
        .left_i(left_i),
        .right_i(right_i),
        .hazard_i(hazard_i),
        .brake_i(brake_i),
        .runlights_i(runlights_i),
        .lights_o(lights)
    );

    initial begin
        rst_n = 1;
        @(posedge clk);
        runlights_i = 0;
        left_i = 0;
        right_i = 0;
        hazard_i = 0;
        brake_i = 0;
        //initial values
    end
endmodule
```

```

brake_i = 1;
@(posedge clk); //brake on

brake_i = 0;
left_i = 1;
repeat (4) @(posedge clk); //left signal

right_i = 1;
repeat (4) @(posedge clk); //left + right signal; => hazard

left_i = 0;
repeat (4) @(posedge clk); //Only right signal

right_i = 0;
hazard_i = 1;
repeat (4) @(posedge clk); // only hazard

brake_i = 1;
hazard_i = 1;
repeat (4) @(posedge clk); // brake + hazard (brake should override)

brake_i = 0;
left_i = 1;
hazard_i = 1;
repeat (4) @(posedge clk); // turn signal + hazard (hazard should
override)

brake_i = 1;
hazard_i = 0;
repeat (4) @(posedge clk); // left turn signal + brake (turn should
override on side)

right_i = 1;
left_i = 0;
repeat (4) @(posedge clk); // right turn signal + brake (turn should
override on side)

rst_n = 0;
repeat (4) @(negedge clk); //setting reset to low, should override
all.

```

```

rst_n = 1;
@(posedge clk);
runlights_i = 0;
left_i = 0;
right_i = 0;
hazard_i = 0;
brake_i = 0;
runlights_i = 1;
//reset initial values + runlights on

brake_i = 1;
@(posedge clk); //brake on

brake_i = 0;
left_i = 1;
repeat (4) @(posedge clk); //left signal

right_i = 1;
repeat (4) @(posedge clk); //left + right signal; => hazard

left_i = 0;
repeat (4) @(posedge clk); //Only right signal

right_i = 0;
hazard_i = 1;
repeat (4) @(posedge clk); // only hazard

brake_i = 1;
hazard_i = 1;
repeat (4) @(posedge clk); // brake + hazard (brake should override)

brake_i = 0;
left_i = 1;
hazard_i = 1;
repeat (4) @(posedge clk); // turn signal + hazard (hazard should
override)

brake_i = 1;
hazard_i = 0;

```

```

    repeat (4) @(posedge clk); // left turn signal + brake (turn should
override on side)

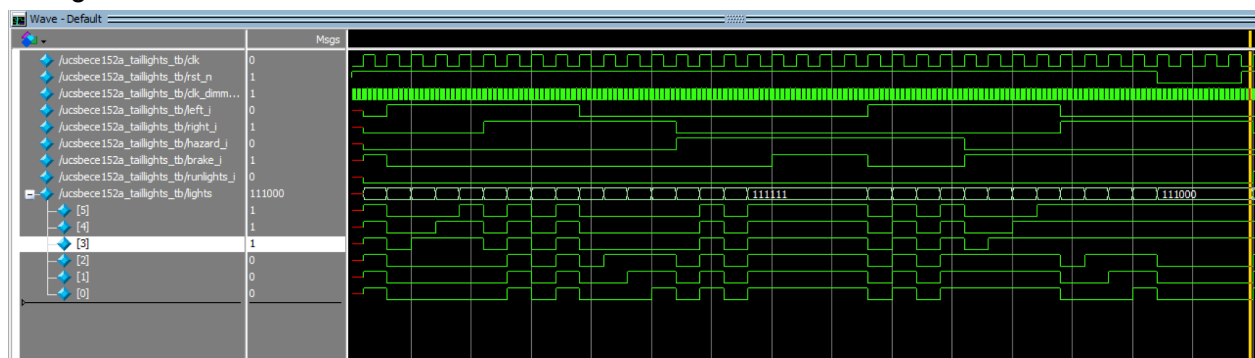
    right_i = 1;
    left_i = 0;
    repeat (4) @(posedge clk); // right turn signal + brake (turn should
override on side)

    rst_n = 0;
    repeat (4) @(negedge clk); //setting reset to low, should override
all.
    $stop;
end
endmodule

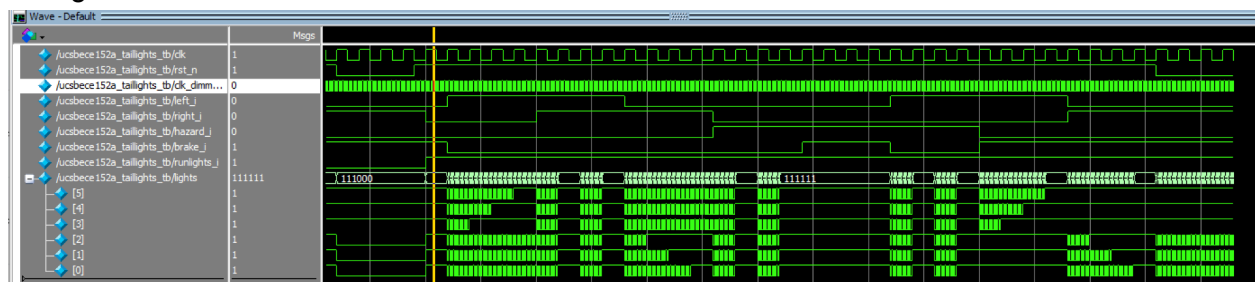
```

Testbench Modelsim Waveform:

Runlights off testcases:



Runlights on testcases:



For runlights off (first image):

The testbench shows that in the beginning the brakes light work as expected with an output of 111111. Then everything else is turned off and left is on, which provides the correct output of 001000-> 011000-> 111000. The right signal is then turned on while the left is also on, which leads to the hazard light function. Left is then turned off which leads to right being the only one on, which leads to the correct output of 000100-> 000110-> 000111. Then everything is turned

off and the hazards are turned on which has the correct output switching between 111111 and 000000. The brakes are then turned on while the hazards are on which overrides the hazards and keeps all the lights on as expected (111111). When the left or right turn signal is on while the hazards are on, only the hazards are displaying as expected. Then while braking, the left and right signals work as expected. Left: (000111->001111->011111->111111) Right: (111000->111100->111110->111111). Finally, we test reset which when set to 0 overrides any outputs received from the FSM.

For runlights on (second image):

The test cases are in the exact same order as for runlights being off. As can be observed, anywhere there would have been a 0 in runlights off is now copying the clk_dimmer_i waveform with runlights on, which when implemented on the board results in dimmer lights. For further explanation of what each test case accomplishes in the order in which they were tested, refer to the paragraph for runlights off.